Residual Deep Lattice Regression for Sparse Multi-Channel DAC Bias Calibration in Beamforming Systems

Sida Xiang
Department of Electrical and Computer Engineering
Rutgers University
Piscataway, NJ, USA
sx178@scarletmail.rutgers.edu

Zhihao Tao
Department of Electrical and Computer Engineering
Rutgers University
Piscataway, NJ, USA
zt118@scarletmail.rutgers.edu

Kailong Wang
Department of Electrical and Computer Engineering
Rutgers University
Piscataway, NJ, USA
kw414@scarletmail.rutgers.edu

Athina P. Petropulu
Department of Electrical and Computer Engineering
Rutgers University
Piscataway, NJ, USA
athinap@soe.rutgers.edu

Minning Zhu*
Department of Electrical and Computer Engineering
Rutgers University
Piscataway, NJ, USA
minning.zhu@rutgers.edu

***Abstract*—Precise bias-voltage generation is critical in digital-to-analog converter (DAC)-controlled beamforming, where phase errors can lead to radiation pattern degradation. Although dense Lookup-Table (LUT) calibration minimizes voltage inaccuracies by exhaustively characterizing every input code, its measurement overhead is unscalable for large-scale arrays or those requires periodic recalibration. In contrast, sparse methods can drastically reduce characterization time by sampling "anchor points" only, a fraction of the code space, to reconstruct the full DAC transfer function analytically. However, simple models such as linear interpolation often fail to capture fine-grained DAC nonlinearity and lead to high reconstruction errors in low-sample-rate regimes. Rather than developing complex custom interpolation mechanisms, we propose residual calibrated lattice regressor (ResDLN) based on a Deep Lattice Network (DLN) that treats the DAC bias realization as a residual learning task. The DLN models nonlinear relationships through a structured combination of calibrators and lattice layers, enabling flexible yet well-regularized function approximation from sparse anchor points. By learning a nonlinear residual correction on top of a baseline linear interpolation using sparse-anchor features, the proposed approach captures fine-grained DAC nonlinearity more effectively than traditional interpolation methods, leading to significant improvement in reconstruction accuracy. Numerical experiments demonstrate that the proposed calibration approach yields superior beam-pattern fidelity compared to conventional interpolation mechanisms.**

***Index Terms*—DAC calibration, beamforming, phased array, deep learning, integrated sensing and communication**

I. Introduction

In bias-controlled RF systems, desired weights are implemented via a digital-to-analog converter (DAC) and analog chain that converts digital commands into specific bias voltages. This hardware stage is foundational to antenna systems [1] such as phased-array beampattern design [2]–[8], time-modulated signal design [9], [10] and ISAC waveform design [11]–[14]. Because system performance depends on implementation accuracy, this work focuses on the hardware realization stage. While DAC codes ideally map to predictable voltages [15], practical multi-channel chains suffer from conversion errors, gain mismatches, and thermal drifts [15], [16]. These non-idealities distort the transfer curve and bias the realized phase-shifter states. Consequently, this paper investigates how to accurately realize a desired bias vector without altering the upstream beamforming solver under non-ideal hardware.

A straightforward solution is dense lookup-table (LUT) calibration [17], where each DAC code is measured and stored for each channel. Dense LUT calibration can be accurate, but it becomes time-consuming when the calibration must be repeated across many channels, boards, sessions, or environmental conditions. Sparse reconstruction-based calibration is a lower-cost alternative: only a small number of anchor points are measured, and the dense transfer curve is reconstructed using linear interpolation, which can be further improved using more advanced interpolation methods such as PCHIP interpolation [18] or cubic spline interpolation [19]. These methods are effective when many anchor points are available.

The difficulty appears when the calibration budget is small. With only a few anchors, the gap between adjacent anchors can be large, and a local interpolation rule may miss nonlinear transfer-curve features such as curvature, saturation knees, or local slope changes. Calibrated lattice models and deep lattice networks (DLN) provide a structured framework for learning

Work supported by NSF under grant ECCS-2433397.

nonlinear input-output functions through feature calibrators and lattice layers [20]–[22]. This motivates the use of a lattice-based residual model for sparse DAC bias calibration. Rather than learning the entire DAC transfer curve from scratch, we propose a residual-calibrated lattice regressor, ResDLN, that corrects the errors left by a sparse linear interpolation baseline.

In the proposed calibration pipeline, each channel and anchor budget is modeled separately. For a given sparse anchor set, linear interpolation first provides a baseline estimate of the dense code-to-voltage curve. ResDLN predicts the baseline residual error by processing the query code and initial estimate alongside local anchor geometry and slope features. The final reconstructed voltage is obtained by adding the predicted residual correction to the linear interpolation estimate. This residual formulation preserves the large-scale trend captured by interpolation while learning structured nonlinear corrections from training data, thereby improving reconstruction accuracy, especially when the anchor points are limited. Numerical results demonstrate that the proposed approach outperforms complex interpolation and improves beamforming performance.

II. System Model and Problem Formulation

The adopted antenna uses four double-phase-shifter (DPS) units [5]–[7], each controlled by two phase-shifter bias voltages, resulting in eight bias channels. This section models the channel-wise DAC-code-to-voltage mapping and formulates sparse calibration as a code-selection problem to accurately realize the desired bias vector.

To formulate this hardware mapping, we index the eight bias channels by $c \in \mathcal{C} = \{1, 2, \ldots, 8\}$. For the 8-bit DAC used in this work, the input code belongs to

$$\mathcal{X} = \{0, 1, \ldots, 255\}. \tag{1}$$

The desired bias vector is denoted by

$$\mathbf{v}^* = [v_1^*, v_2^*, \ldots, v_8^*]^\mathsf{T}, \tag{2}$$

where v_c^* is the desired bias voltage for channel c. For channel c, let $y_c(x)$ denote the final output voltage after DAC conversion and analog conditioning when DAC code $x \in \mathcal{X}$ is applied. The set of values $\{y_c(x) : x \in \mathcal{X}\}$ represents the dense code-to-voltage transfer curve of channel c.

If the dense transfer curve $y_c(x)$ is fully known for all $x \in \mathcal{X}$, the dense-LUT oracle selects the code that minimizes the voltage error with respect to the desired bias voltage:

$$x_c^{\text{oracle}} = \arg\min_{x \in \mathcal{X}} |y_c(x) - v_c^*| . \tag{3}$$

The corresponding oracle realized voltage is

$$v_c^{\text{oracle}} = y_c\big(x_c^{\text{oracle}}\big) . \tag{4}$$

This dense-LUT solution is used only as a reference because it requires characterizing all DAC codes for every channel.

Under K-sparse calibration, only K DAC codes are sampled from the full 256-code range. Let $\mathcal{S}_K \subset \mathcal{X}$ denote the selected K-point anchor-code set, where $|\mathcal{S}_K| = K$. For channel c, the sparse calibration data are defined as

$$\mathcal{A}_c^{(K)} = \{(x, y_c(x)) : x \in \mathcal{S}_K\} . \tag{5}$$

Each pair in $\mathcal{A}_c^{(K)}$ contains one sampled DAC code and its corresponding output voltage. The anchor-code set $\mathcal{S}_K$ may be selected according to a measurement-design rule. The validation-guided greedy procedure used in the experiments is described in Section IV-B.

Let $m \in \mathcal{M}$ denote a sparse calibration method. The compared methods are linear interpolation, PCHIP interpolation [18], cubic spline interpolation [19], and the proposed residual DLN, denoted ResDLN. Thus,

$$\mathcal{M} = \{\text{linear}, \text{PCHIP}, \text{spline}, \text{ResDLN}\}. \tag{6}$$

Each method reconstructs a dense estimate

$$\hat{y}_c^{(m)}(x), \quad x \in \mathcal{X}, \tag{7}$$

from the sparse anchor set $\mathcal{A}_c^{(K)}$. The selected code under method m is then

$$\hat{x}_c^{(m)} = \arg\min_{x \in \mathcal{X}} \left| \hat{y}_c^{(m)}(x) - v_c^* \right| . \tag{8}$$

The realized voltage is evaluated using the dense truth curve, not the reconstructed curve:

$$v_c^{(m)} = y_c\Big(\hat{x}_c^{(m)}\Big) . \tag{9}$$

For all eight channels, the realized voltage vector is

$$\mathbf{v}^{(m)} = [v_1^{(m)}, v_2^{(m)}, \ldots, v_8^{(m)}]^\mathsf{T}. \tag{10}$$

Thus, sparse calibration aims to reconstruct the channel-wise code-to-voltage curves accurately enough to select DAC codes whose true realized voltages are close to the desired bias vector.

III. Methodology

In this section, we describe the proposed ResDLN sparse-calibration method, including the residual formulation, input features, calibrated-lattice model architecture, training objective, and testing protocol.

A. ResDLN Calibration Model

The proposed method is a residual calibrated lattice regressor based on DLN. It does not directly learn the full DAC transfer curve from sparse anchors. Instead, it first uses sparse linear interpolation as a baseline and then learns a residual correction to this baseline. This residual formulation reduces the learning burden because the linear baseline already captures the large-scale code-to-voltage trend.

Let $\hat{y}_c^{\text{lin}}(x)$ denote the sparse linear interpolation estimate obtained from $\mathcal{A}_c^{(K)}$. The residual target for channel c is

$$r_c(x) = y_c(x) - \hat{y}_c^{\text{lin}}(x). \tag{11}$$

Let $f_{c,K}(\cdot)$ denote the ResDLN trained for channel c and anchor count K. The residual prediction is

$$\hat{r}_c(x) = f_{c,K}\Big(\boldsymbol{\phi}_c^{(K)}(x)\Big) , \tag{12}$$

where $\boldsymbol{\phi}_c^{(K)}(x)$ is the input feature vector associated with code x. The final ResDLN reconstruction is

$$\hat{y}_c^{\mathrm{ResDLN}}(x) = \hat{y}_c^{\mathrm{lin}}(x) + \hat{r}_c(x). \tag{13}$$

The feature vector $\boldsymbol{\phi}_c^{(K)}(x)$ describes the local interpolation context around the query code. It includes the normalized query code, the sparse linear estimate, neighboring-anchor locations and voltages, distances to neighboring anchors, local slope information, and high-code features that indicate saturation-like behavior. All distance and slope features are computed using the normalized code

$$z = \frac{x}{255}, \quad z \in [0, 1], \tag{14}$$

and boundary codes use the first or last valid anchor segment.

The model follows the calibrated-lattice framework [20]–[22]. Each scalar feature passes through a piecewise linear calibration layer with 8 training-range keypoints. The calibrated outputs are then passed to a 12-dimensional lattice layer with two vertices per input dimension, producing a scalar residual correction. No monotonicity constraint is imposed, so the model is a residual calibrated lattice regressor rather than a monotonicity-constrained DLN.

B. Training Objective

An independent ResDLN is trained for each channel c and each anchor count K. No cross-channel shared model is used. Let $\mathcal{D}_{\mathrm{tr}}$ denote the training set for a fixed channel and anchor count. For each training sample $q \in \mathcal{D}_{\mathrm{tr}}$, let r_q be the residual target and let $\boldsymbol{\phi}_q$ be the corresponding input feature vector. We employ an unweighted mean-absolute-error (MAE) loss:

$$\mathcal{L}_{c,K} = \frac{1}{|\mathcal{D}_{\mathrm{tr}}|} \sum_{q \in \mathcal{D}_{\mathrm{tr}}} \left| r_q - f_{c,K}(\boldsymbol{\phi}_q) \right|, \tag{15}$$

and an Adam [23] optimizer with an initial learning rate of 0.005, batch size 1024, and at most 90 epochs. Validation loss is used for early stopping and for learning rate reduction.

IV. Experiments Setup

The proposed residual-learning formulation requires dense or sufficiently high-resolution ground-truth transfer curves during offline training. This is because the residual target is computed as the difference between the true voltage and the sparse linear-interpolation estimate (Eq. (11)). Therefore, the dense curves are used to generate supervised residual labels for training and validation. Importantly, this dense characterization is required only during the offline model development stage. After the residual model has been trained, deployment on a new calibration session only requires the sparse anchor measurements, from which the linear baseline, local features, and ResDLN residual correction can be computed.

A. Simulated Sparse-Calibration Dataset

The experiments use simulated dense DAC-and-analog transfer curves as ground truth. Each simulation session represents one realization of a channel-dependent code-to-voltage transfer curve, including gain and offset variation, nonlinear conversion behavior, and high-code saturation effects. For each anchor budget K, the sparse anchor set is selected from the 256-code DAC range using the validation-guided greedy procedure described in Section IV-B. The anchor budgets are

$$K \in \{8, 12, 16, 24, 32\}. \tag{16}$$

For the all-channel $K = 8$ experiment, each channel contains 160 simulated sessions. The data are split by session ID into 112 training sessions, 24 validation sessions, and 24 test sessions. Since each session contains 256 DAC-code samples, this corresponds to 28,672 training rows, 6,144 validation rows, and 6,144 test rows per channel. This session-level split prevents samples from the same simulated transfer curve from appearing in multiple splits.

B. Validation-Guided Greedy Anchor Selection

The sparse anchor locations are selected using an offline validation-guided greedy procedure. This step only determines the DAC codes to be measured under a fixed anchor budget and is not part of the ResDLN architecture. For each target budget K, the procedure starts from the two endpoint codes,

$$\mathcal{S}_2 = \{0, 255\}. \tag{17}$$

Candidate anchors are selected from the interior code set

$$\mathcal{X}_{\mathrm{int}} = \{1, 2, \ldots, 254\}. \tag{18}$$

Let $\mathcal{V}$ denote the set of validation transfer curves, where each curve is indexed by channel c and validation session s. We denote the corresponding dense truth curve by $y_{c,s}(x)$, $x \in \mathcal{X}$. For a candidate anchor set $\mathcal{S}$, let $\hat{y}_{c,s}^{\mathrm{lin}}(x; \mathcal{S})$ be the dense curve reconstructed from the samples at the anchor codes in $\mathcal{S}$ using linear interpolation. The validation reconstruction error is defined as

$$E_{\mathrm{val}}(\mathcal{S}) = \frac{1}{|\mathcal{V}|\,|\mathcal{X}|} \sum_{(c,s) \in \mathcal{V}} \sum_{x \in \mathcal{X}} \left| y_{c,s}(x) - \hat{y}_{c,s}^{\mathrm{lin}}(x; \mathcal{S}) \right|. \tag{19}$$

At step $t = 3, \ldots, K$, each unselected DAC code is tested as the next candidate anchor. The code that gives the smallest validation reconstruction error is selected:

$$x_t^\star = \underset{x \in \mathcal{X}_{\mathrm{int}} \setminus \mathcal{S}_{t-1}}{\arg\min} \; E_{\mathrm{val}}\left(\mathrm{sort}\left(\mathcal{S}_{t-1} \cup \{x\}\right)\right), \tag{20}$$

and the anchor set is updated as

$$\mathcal{S}_t = \mathrm{sort}\left(\mathcal{S}_{t-1} \cup \{x_t^\star\}\right). \tag{21}$$

This process is repeated until K anchors are selected. The held-out test sessions are not used during this selection process. After selection, the anchor set is fixed for the given K, shared across all eight channels, and used by all sparse reconstruction methods.

For the final $K = 8$ evaluation, the selected anchor set is

$$\mathcal{S}_8^{\text{greedy}} = \{0, 69, 131, 198, 211, 221, 230, 255\}. \tag{22}$$

The selected codes are denser in the high-code region, where the simulated transfer curves include stronger nonlinear and saturation-like behavior. This is consistent with the validation objective, which assigns more anchors to code regions that are more difficult to reconstruct from sparse samples.

C. Evaluation Methodology and Metrics

The proposed ResDLN is compared against three baseline reconstruction methods—linear, PCHIP, and cubic spline interpolation—using a common K-point anchor set sampled from noise-free simulated truth curves. During testing, each method reconstructs the dense code-to-voltage transfer curve from these sparse anchors. For fixed-bias validation, the reconstructed curves are used to select DAC codes according to Eq. (8), while the final realized voltage is determined from the dense truth curve per Eq. (9). This procedure ensures that each method is evaluated based on its physical implementation accuracy rather than on its predicted voltages, preventing self-evaluation bias.

In a practical scenario, dense measurements would first be collected offline from representative devices, channels, and environmental conditions to train the residual model. For a future device or recalibration session, only the K sparse anchor voltages are measured. These anchors are used to construct the sparse linear-interpolation baseline and the local feature vector for each query code. The trained ResDLN then predicts the residual correction across the DAC code range, producing a reconstructed dense transfer curve. This reconstructed curve is subsequently used to select the DAC code whose predicted realized voltage is closest to the desired bias voltage.

D. Fixed-Bias Beam-Pattern Numerical Validation

For system-level validation, we use the ideal eight-channel bias vector (in volts (V))

$$\mathbf{v}^* = [3.8, 4.8, 4.6, 1.0, 6.4, 12.2, 9.4, 6.2]^{\mathsf{T}}. \tag{23}$$

Following the radar-based vital-sign monitoring and automated beam-steering framework in [24], the upstream beamforming solution is fixed and not re-optimized. The same desired bias vector is realized using linear interpolation, PCHIP interpolation, cubic spline interpolation, and ResDLN, and the resulting voltage vectors are evaluated by the same downstream numerical beam-pattern model. The beam-pattern comparison is used solely for numerical forward validation and should not be interpreted as a measured antenna pattern.

V. Simulation Results

A. Sparse Calibration Accuracy versus Calibration Budget

We show the pooled all-channel reconstruction MAE under the validation-guided greedy anchor configuration as a function of the number of sparse calibration anchors in Fig. 1. The comparison highlights the effect of adding the proposed residual correction to a sparse linear interpolation baseline. ResDLN provides its largest improvement at low calibration budgets,

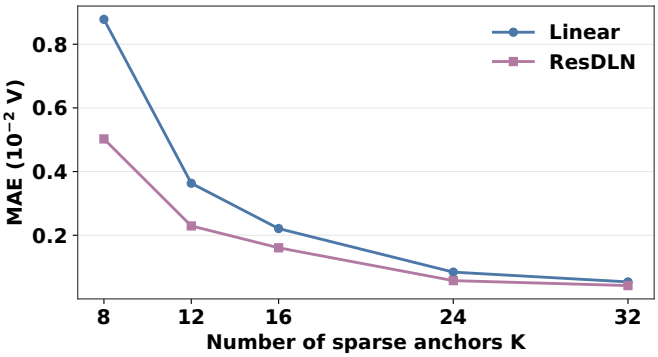


Fig. 1. Pooled all-channel sparse-calibration MAE versus the number of calibration anchors under the validation-guided greedy anchor configuration. ResDLN improves over the linear baseline, especially in the low-K regime.

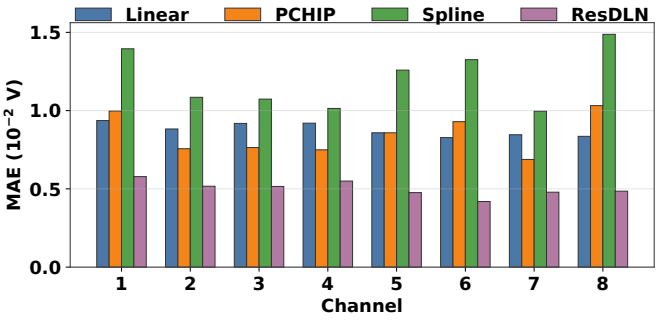


Fig. 2. All-channel simulation at $K = 8$ under the validation-guided greedy anchor configuration: channel-wise overall MAE for linear interpolation, PCHIP interpolation, cubic spline interpolation, and ResDLN.

especially at $K = 8$, where the sparse anchors are too widely spaced for linear interpolation to capture nonlinear transfer-curve behavior. As K increases, the anchor spacing decreases, and the linear baseline becomes more competitive, so the performance gap narrows. This result supports the use of ResDLN mainly in the low-sample calibration regime.

B. All-Channel Low-Budget Result at K=8

The $K = 8$ case is the most challenging calibration setting considered in this work. We summarize the channel-wise overall reconstruction MAE for all eight channels in Fig. 2. ResDLN achieves the lowest overall MAE across channels 1–8. This result indicates that the residual formulation is effective not only for a single transfer curve, but also across channel-dependent simulated transfer characteristics.

To quantify the pooled all-channel improvement, we compute the MAE reduction of ResDLN relative to each baseline b as

$$R_b = \frac{\text{MAE}^{(b)} - \text{MAE}^{(\text{ResDLN})}}{\text{MAE}^{(b)}} \times 100\%, \tag{24}$$

where $b \in \{\text{linear}, \text{PCHIP}, \text{spline}\}$. Using this definition, ResDLN reduces the pooled all-channel MAE by 42.77%, 40.66%, and 58.28% relative to linear interpolation, PCHIP interpolation, and cubic spline interpolation, respectively. These

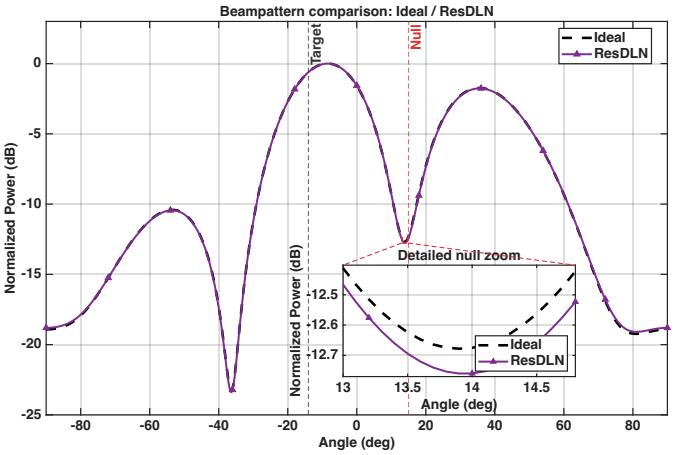


Fig. 3. Beam-pattern comparison for the fixed-bias numerical validation under the $K = 8$ greedy anchor configuration. The beam pattern produced by the ResDLN-realized bias vector closely follows an ideal-bias reference. A zoomed view of the null region shows that the null response is also well preserved.

results support the main claim that ResDLN is most useful when the calibration budget is very small.

C. Fixed-Bias Voltage and Beam-Pattern Validation

We next evaluate whether the voltage-level improvement translates to the fixed-bias beam-pattern validation case. For the $K = 8$ fixed-bias experiment under the greedy anchor configuration, ResDLN produces the closest realized voltage vector among the tested sparse methods. In terms of the average absolute voltage error over the eight bias channels, ResDLN reduces the error by 3.12% relative to PCHIP and by 8.45% relative to linear interpolation.

The realized voltage vectors are then evaluated using the numerical beam-pattern forward model. The comparison uses the same ideal bias vector and the same downstream phase-shifter and array model, so the observed pattern differences are due solely to differences in sparse DAC bias realization. We compare the ideal beam pattern with the beam pattern obtained from the ResDLN-realized bias vector and show in Fig. 3. The two responses are closely matched over the main angular range, and the inset further shows that the null region is preserved with only a small deviation from the ideal response.

These beam-pattern results should be interpreted as numerical forward validation rather than measured antenna-pattern validation. The purpose is to quantify how bias-realization error propagates through a fixed downstream beam-pattern model, not to claim measured antenna improvement on physical hardware.

VI. Conclusion

This paper formulated multi-channel DAC bias realization as a sparse calibration problem for bias-controlled beamforming systems. The proposed ResDLN approach uses sparse linear interpolation as a baseline and learns a residual correction with a calibrated lattice regressor. Simulation results under the validation-guided greedy anchor configuration show that ResDLN performs best with a small number of anchors, especially at $K = 8$, where it significantly reduces pooled all-channel MAE compared to linear, PCHIP, and cubic spline interpolation. Fixed-bias numerical validation further shows that ResDLN produces the closest realized voltage vector among the tested sparse methods and yields a beam pattern close to the ideal-bias reference. The expected benefit of the proposed approach depends on how well the offline training curves represent the devices and operating conditions encountered during deployment. If a future board, DAC, analog buffer, load condition, or temperature range differs substantially from the training distribution, the learned residual correction may become less reliable. In such cases, additional dense or semi-dense measurements may be required for adaptation, fine-tuning, or retraining. Our future work will extend the adaptability of this method with measured hardware validation and real-data adaptation considering board-level and more environmental variations.

References

[1] C. A. Balanis, *Antenna Theory: Analysis and Design*, 4th ed. Hoboken, NJ, USA: Wiley, 2016.

[2] R. C. Hansen, *Phased Array Antennas*, 2nd ed. Hoboken, NJ, USA: Wiley, 2009.

[3] M. Zhu, T.-W. Kuo, and C.-T. M. Wu, "A reconfigurable linear rf analog processor for realizing microwave artificial neural network," *IEEE Transactions on Microwave Theory and Techniques*, vol. 72, no. 2, pp. 1290–1301, 2024.

[4] Z. Xu, C. Shi, T. Zhang, S. Li, Y. Yuan, C.-T. M. Wu, Y. Chen, and A. P. Petropulu, "Simultaneous monitoring of multiple people's vital sign leveraging a single phased-MIMO radar," *IEEE Journal of Electromagnetics, RF and Microwaves in Medicine and Biology*, vol. 6, no. 3, pp. 311–320, Sept. 2022.

[5] Z. Xu and A. P. Petropulu, "Phased array with improved beamforming capability via use of double phase shifters," in *Proc. IEEE 12th Sensor Array and Multichannel Signal Processing Workshop (SAM)*, 2022, pp. 66–70.

[6] Z. Xu, D. Gao, S. Li, C.-T. M. Wu, and A. P. Petropulu, "Flexible beam design for vital sign monitoring using a phased array equipped with double-phase shifters," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[7] ——, "Phased array with non-ideal double phase shifters for multi-target vital sign monitoring," in *Proc. IEEE Int. Symp. Biomedical Imaging (ISBI)*, 2024, pp. 1–4.

[8] Z. Tao and A. P. Petropulu, "Antenna selection for enhancing privacy in radar-based vital sign monitoring systems," in *Proc. IEEE Int. Radar Conf. (RADAR)*, 2025.

[9] ——, "Meta-learning-driven gflownets for 3d directional modulation in mobile wireless systems," in *Proc. IEEE ICC 2026*, Glasgow, Scotland, UK, 2026.

[10] ——, "Secure time-modulated intelligent reflecting surface via generative flow networks," in *Proc. IEEE MILCOM 2025*, Los Angeles, CA, USA, 2025.

[11] Z. Tao, A. Petropulu, and H. V. Poor, "Time-modulated intelligent reflecting surfaces for integrated sensing, communication and security: A generative ai design framework," *npj Wireless Technology*, March 2026, in minor revision.

[12] K. Wang and A. Petropulu, "Isac mimo systems with otfs waveforms and virtual arrays," *IEEE Journal on Selected Areas in Communications*, vol. 44, pp. 229–244, 2026.

[13] ——, "Low overhead and scalable time-frequency pilots design for mimo otfs channel estimation," 2026. [Online]. Available: https://arxiv.org/abs/2511.08504

[14] ——, "Virtual array for dual function mimo radar communication systems using otfs waveforms," in *2025 IEEE 5th International Symposium on Joint Communications & Sensing (JC&S)*, 2025, pp. 1–6.

[15] *AD7303: +2.7 V to +5.5 V, Serial Input, Dual Voltage Output 8-bit DAC*, Analog Devices, 1997, data Sheet.

[16] *LM741 Single Operational Amplifier*, Fairchild Semiconductor, 2001, data Sheet.

[17] K. Parthasarathy, T. Kuyel, Z. Yu, D. Chen, and R. L. Geiger, "A 16-bit resistor string DAC with full-calibration at final test," in *Proc. IEEE International Test Conference (ITC)*, Austin, TX, USA, Nov. 2005, pp. 66–75.

[18] F. N. Fritsch and R. E. Carlson, "Monotone piecewise cubic interpolation," *SIAM Journal on Numerical Analysis*, vol. 17, no. 2, pp. 238–246, Apr. 1980.

[19] C. de Boor, *A Practical Guide to Splines*. New York, NY, USA: Springer, 2001.

[20] M. Gupta, A. Cotter, J. Pfeifer, K. Voevodski, K. Canini, A. Mangylov, W. Moczydlowski, and A. van Esbroeck, "Monotonic calibrated interpolated look-up tables," *Journal of Machine Learning Research*, vol. 17, no. 109, pp. 1–47, 2016.

[21] S. You, D. Ding, K. Canini, J. Pfeifer, and M. R. Gupta, "Deep lattice networks and partial monotonic functions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[22] M. Milani Fard, K. Canini, A. Cotter, J. Pfeifer, and M. R. Gupta, "Fast and flexible monotonic functions with ensembles of lattices," in *Advances in Neural Information Processing Systems*, vol. 29, 2016.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[24] D. Gore, D. Petronchak, F. Valencia, G. Young, N. Warren, and A. Petropulu, "Radar-based vital sign monitoring with automated beam steering," *Journal of Engineering Technology*, vol. 42, no. 1, pp. 34–43, 2025.